

Medidas de desempeño & cross-entropy & softmax

curso
Deep Learning

Prof. M.A.Valle
E-MAIL: mauricio.valle.b@uai.cl

Métricas de evaluación (clasificación)

Evaluar un modelo, es necesario revisar si los valores pronosticados (**predicted values**) son iguales a los valores actuales o reales (**actual values**).

Tenemos 4 posibilidades:

Predicted	Actual
T	T
T	F
F	T
F	F

Métricas de evaluación (clasificación)

Evaluar un modelo, es necesario revisar si los valores pronosticados (**predicted values**) son iguales a los valores actuales o reales (**actual values**).

Tenemos 4 posibilidades:

Predicted	Actual
T	T
T	F
F	T
F	F

Ovbiamente, nos interesa que predicted y actual values sean iguales, es decir, F-F o T-T.

Matriz de confusión

Podemos tabular y contar estas cuatro posibilidades para un problema de clasificación binaria, a partir de la cual calculamos nuestras métricas de evaluación.

	Predicted value	
Actual Values	T	F
T	TP	FN
F	FP	TN

Un buen modelo clasificador, obtendrá gran cantidad de TP y/o de TN, mientras que un mal clasificador tendrá mucha cantidad de FP y/o FN.

Matriz de confusión

Accuracy

	Predicted value		
Actual Values	T	F	
T	TP	FN	P
F	FP	TN	N

$$ACC = (TP + TN) / (P + N) = (TP + TN) / (TP + TN + FP + FN)$$

Es el ratio entre las predicciones correctas y el total de predicciones.

Matriz de confusión

Precision o True Positive rate o sensitivity

	Predicted value		
Actual Values	T	F	
T	TP	FN	P
F	FP	TN	N

$$PRE_+ = TP / (TP + FP)$$

Es el ratio entre las predicciones correctas “positivas” y el total de positivos identificados por la máquina.

Matriz de confusión

Recall o True Negative rate o specificity

	Predicted value		
Actual Values	T	F	
T	TP	FN	P
F	FP	TN	N

$$REC_+ = TP / P = TP / (TP + FN)$$

Es el ratio entre las predicciones correctas “positivas” y el total de positivos actuales.

Matriz de confusión

F-score

	Predicted value		
Actual Values	T	F	
T	TP	FN	P
F	FP	TN	N

$$F_{+} = 2 * (PRE * REC) / (PRE + REC)$$

El F-score no es nada más que la media armónica del precision y del recall.

Binary-cross entropy

Para clasificadores binarios, utilizamos como función de pérdida el **binary cross-entropy**.

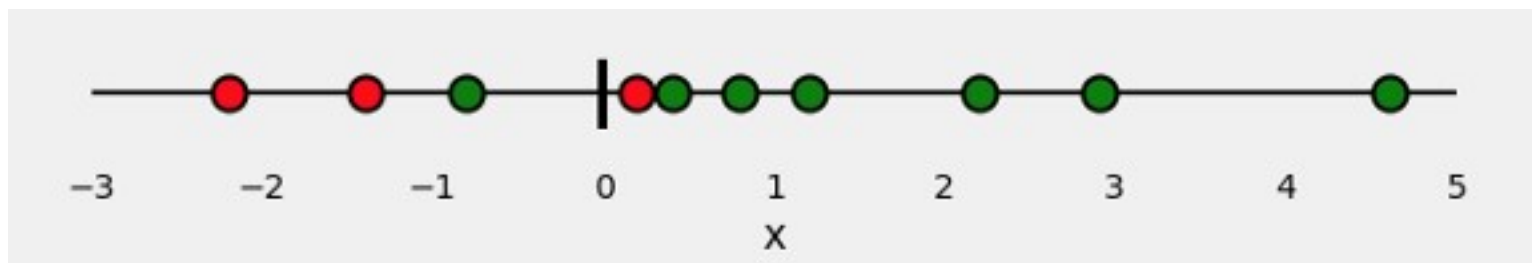
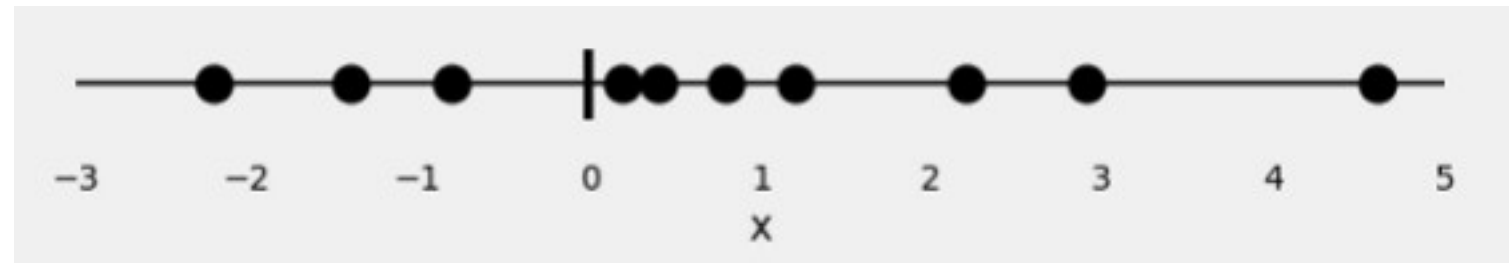
Esta es una función que se utiliza mucho para clasificadores y que ya viene incorporada en varias librerías en R y Python.

Pero, en qué consiste realmente esta función?

Binary-cross entropy

Supongamos un problema de clasificación con sólo 1 atributo:

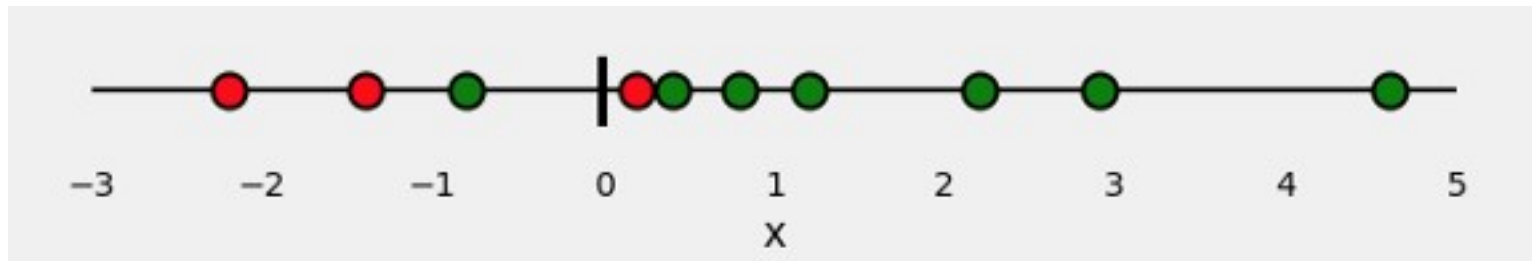
$$x = [-2.2, -1.4, -0.8, 0.2, 0.4, 0.8, 1.2, 2.2, 2.9, 4.6]$$



Nuestro problema de clasificación consiste en discriminar verdes (YES , 1) de rojos (NO, 0) con solo el atributo x.

Binary-cross entropy

Lo que debe hacer el clasificador es predecir la probabilidad de que sea verde (o rojo), por ejemplo, $P(y_i = \text{verde} | X)$ para cada uno de los puntos.



Como ya sabemos el color, ¿cómo podemos evaluar cuán mal o bien estas estas probabilidades?

Esa es la idea de la función de pérdida.

Retornará valores grandes para malas predicciones y valores bajo para buenas predicciones.

Así, para cada punto y_i , el modelo nos dirá la probabilidad de que sea verde o rojo, y en función de aquello evaluar la función de pérdida.

Binary-cross entropy

Definición de binary cross-entropy:

$$H_p(q) = -\frac{1}{N} \sum_{i=1}^N y_i \cdot \log(p(y_i)) + (1 - y_i) \cdot \log(1 - p(y_i))$$

Donde y_i es la clase (1 para verde, 0 para rojo) a la cual pertenece la instancia 'i', y $p(y_i)$ es la predicted probability de que el punto sea verde para todos los N puntos.

Básicamente, H es una entropía promedio.

Binary-cross entropy

Definición de binary cross-entropy:

$$H_p(q) = -\frac{1}{N} \sum_{i=1}^N y_i \cdot \log(p(y_i)) + (1 - y_i) \cdot \log(1 - p(y_i))$$

Supongamos que $y_i = 1$ (**verde**), y que $p(y_i=\text{verde}) = 0.99$, en ese caso

$$H = - (1 \cdot \log(0.99) + 0) = +0.0043$$

Supongamos que $y_i = 0$ (**rojo**), y que $p(y_i=\text{verde}) = 0.01$, en ese caso

$$H = - (0 + 1 \cdot \log(0.99)) = +0.0043$$

Binary-cross entropy

Definición de binary cross-entropy:

$$H_p(q) = -\frac{1}{N} \sum_{i=1}^N y_i \cdot \log(p(y_i)) + (1 - y_i) \cdot \log(1 - p(y_i))$$

Supongamos que $y = 1$ (**verde**), y que $p(y_i=\text{verde}) = 0.01$, en ese caso

$$H = -(1 \cdot \log(0.01) + 0) = +2$$

Supongamos que $y = 0$ (**rojo**), y que $p(y_i=\text{verde}) = 0.99$, en ese caso

$$H = -(0 + 1 \cdot \log(1-0.99)) = +2$$

Binary-cross entropy

Definición de binary cross-entropy:

$$H_p(q) = -\frac{1}{N} \sum_{i=1}^N y_i \cdot \log(p(y_i)) + (1 - y_i) \cdot \log(1 - p(y_i))$$

Como podemos ver, cuando el clasificador se equivoca mucho, el valor de H se vuelve muy grande positivo..

Mientras que cuando el clasificador no se equivoca, el valor de H se vuelve pequeño cercano a cero.

La idea es que H sea lo más pequeño posible!

Softmax (para problemas multiclass)

En los casos en que utilizamos una NN para actividades de clasificación de más de 2 labels (multiclass), la función de activación sigmoid (o `hard_sigmoid`) no nos sirve.

(nota: `hard_sigmoid` en keras es una aproximación lineal a la sigmoide para evitar el cómputo de la exponencial en la derivadas, por lo que se ahorra bastante tiempo, puede usar `hard_sigmoid` en caso que sea necesario.).

Por ejemplo, para un problema de clasificación de 4 clases, quisiéramos tener un vector final del tipo `[0.02, 0, 0.005, 0.975]` el cual podemos **interpretar como la probabilidad de que la instancia pertenezca a cada una de las clases**.

Esto último será sólo posible si ponemos al final, una capa de salida con función de activación SOFTMAX y utilizamos una versión modificada de binary-cross entropy como función de pérdida: **`categorical_crossentropy`**.

Softmax (para problemas multiclass)

Recordemos que una Distribución Bernoulli es una pdf discreta que nos sirve para modelar el outcome de una va de un experimento con dos posibilidades (ej. si / no). Pero también nos sirve para modelar pdf's de va's con experimentos con más de 2 posibilidades.

Recuerda regresión logística?

Asume que la slida Y_i condicional a x_i se distribuye:

$$Y_i | x_{1,i}, \dots, x_{m,i} \sim \text{Bernoulli}(p_i)$$
$$E(Y_i | x_{1,i}, \dots, x_{m,i}) = p_i$$
$$\Pr(Y_i = y | x_{1,i}, \dots, x_{m,i}) = \begin{cases} p_i & \text{if } y = 1 \\ 1 - p_i & \text{if } y = 0 \end{cases}$$

Softmax (para problemas multiclass)

La función que relaciona el log odds de resultados distribuidos Bernoulli con el predictor lineal es la función **logit**:

$$\begin{aligned}\text{logit}(\mathbb{E}[Y_i | x_{1,i}, \dots, x_{m,i}]) &= \text{logit}(p_i) \\ &= \ln \left(\frac{p_i}{1 - p_i} \right) = \beta_0 + \beta_1 x_{1,i} + \dots + \beta_m x_{m,i}\end{aligned}$$

Si aplicamos exponencial a ambos lados de la ecuación, y arreglamos un poco el RHS, obtenemos la expresión que todos conocemos:

$$\begin{aligned}\mathbb{E}[Y_i | \mathbf{X}_i] &= p_i = \text{logit}^{-1}(\beta \mathbf{X}_i) \\ &= \frac{1}{1 + e^{-\beta \mathbf{X}_i}}\end{aligned}$$

Softmax (para problemas multiclass)

¿ y qué tiene que ver esto con SOFTMAX?

Consideremos una regresión multinomial logística, para predecir cada una de las K clases, más un factor de normalización para que las probabilidades sumen 1.

Así para cada clase tendríamos:

$$\begin{aligned}\ln \Pr(Y_i = 1) &= \beta_1 \mathbf{X}_i - \ln Z \\ \ln \Pr(Y_i = 2) &= \beta_2 \mathbf{X}_i - \ln Z \\ &\dots\dots\dots \\ \ln \Pr(Y_i = K) &= \beta_K \mathbf{X}_i - \ln Z\end{aligned}$$

Donde $\ln(Z)$ el log natural de factor de normalización conocido como **función de partición**, el cual se puede escribir como:

$$Z = \sum_{k=1}^K \exp(\beta_k \mathbf{X}_i)$$

Softmax (para problemas multiclass)

Entonces, podemos escribir de la ecuación anterior que:

$$\begin{aligned}\Pr(Y_i = 1) &= \frac{\exp(\beta_1 \mathbf{X}_i)}{\sum_{k=1}^K \exp(\beta_k \mathbf{X}_i)} \\ &\dots \\ \Pr(Y_i = K) &= \frac{\exp(\beta_K \mathbf{X}_i)}{\sum_{k=1}^K \exp(\beta_k \mathbf{X}_i)}\end{aligned}$$

La división del RHS de cada ecuación es la función **SOFTMAX**!

Como podemos ver, la función normaliza un vector dimensional X de valores reales en un vector K -dimensional $\Pr(Y=K)$ cuyos componentes deben sumar 1 (es decir, es una probabilidad).

Por supuesto, en nuestro caso de redes neuronales, los parámetros betas, serían los pesos de las redes sinápticas que llegan a la capa de salida.